

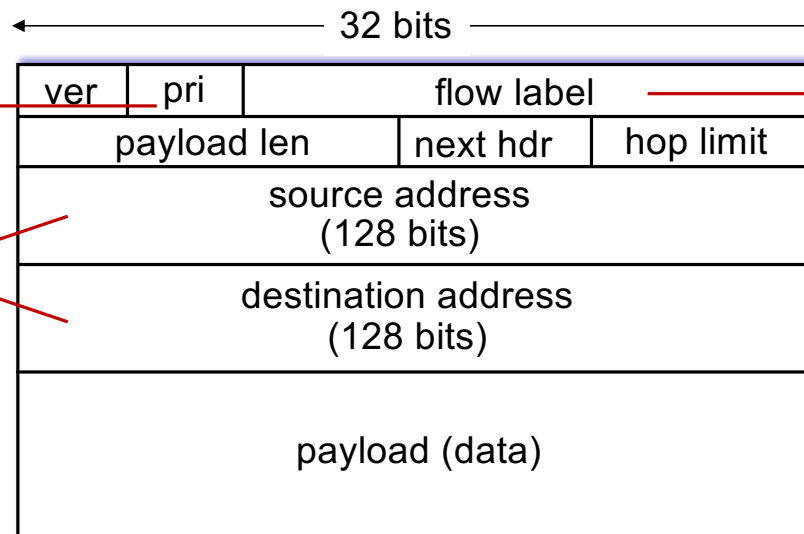
IPv6: motivation

- **initial motivation:** 32-bit IPv4 address space would be completely allocated
- additional motivation:
 - speed processing/forwarding: 40-byte fixed length header
 - enable different network-layer treatment of “flows”

IPv6 datagram format

priority: identify priority among datagrams in flow

128-bit
IPv6 addresses



flow label: identify datagrams in same "flow." (concept of "flow" not well defined).

What's missing (compared with IPv4):

- no checksum (to speed processing at routers)
- no fragmentation/reassembly
- no options (available as upper-layer, next-header protocol at router)

Design Philosophy of IPv6

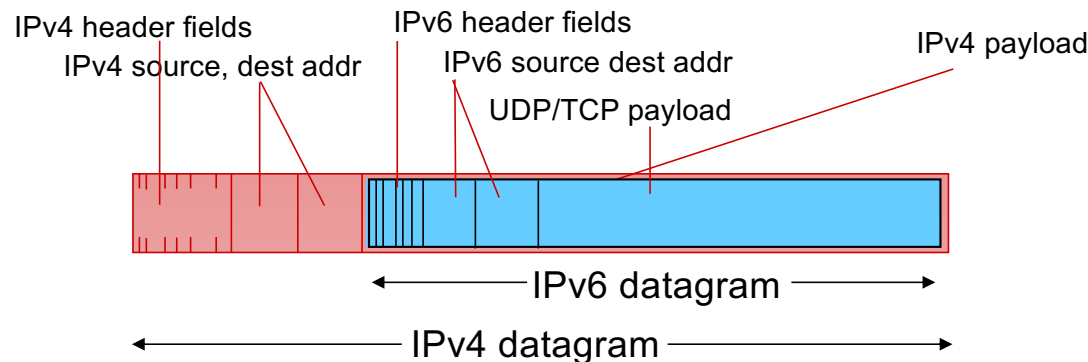
- Don't deal with problems: leave to ends
 - Eliminated fragmentation
 - Eliminated checksum
 - *Take a look at a classic paper¹ on end-to-end argument*
- Simplify handling
 - New options mechanism (uses next header approach)
 - Eliminated header length
- Provide general flow label for packet
 - Not tied to semantics
 - Provides great flexibility

¹End-to-end argument in systems design, 1984.

<http://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>

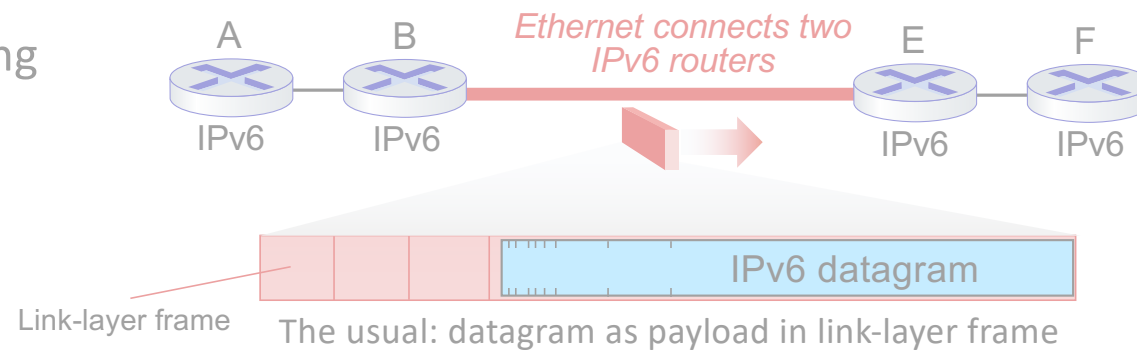
Transition from IPv4 to IPv6

- not all routers can be upgraded simultaneously
 - no “flag days”
 - how will network operate with mixed IPv4 and IPv6 routers?
- **tunneling**: IPv6 datagram carried as *payload* in IPv4 datagram among IPv4 routers (“packet within a packet”)
 - tunneling used extensively in other contexts (4G/5G)

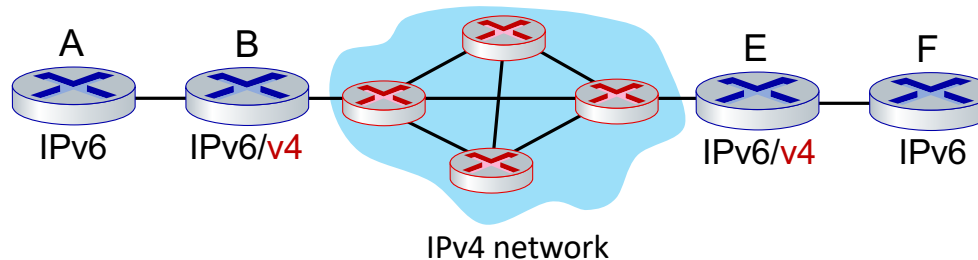


Tunneling and encapsulation

Ethernet connecting two IPv6 routers:

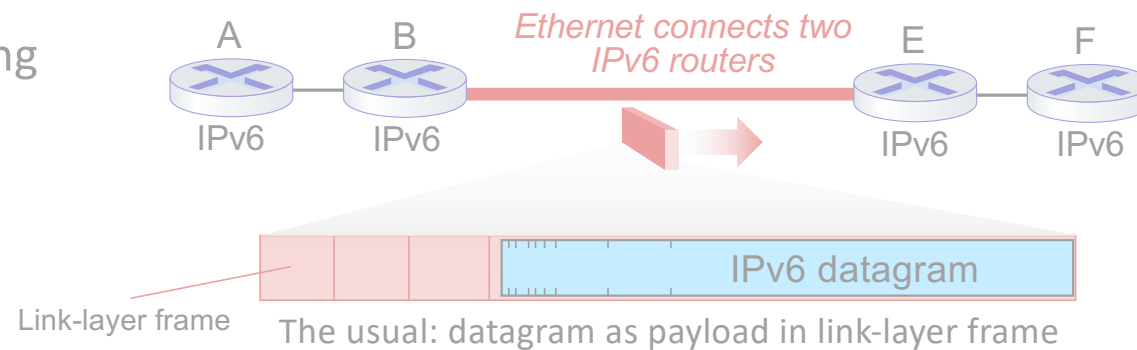


IPv4 network connecting two IPv6 routers

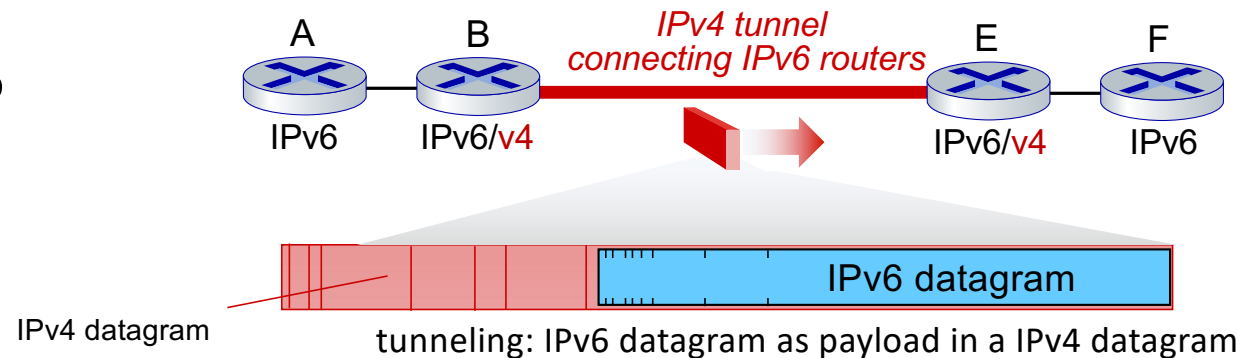


Tunneling and encapsulation

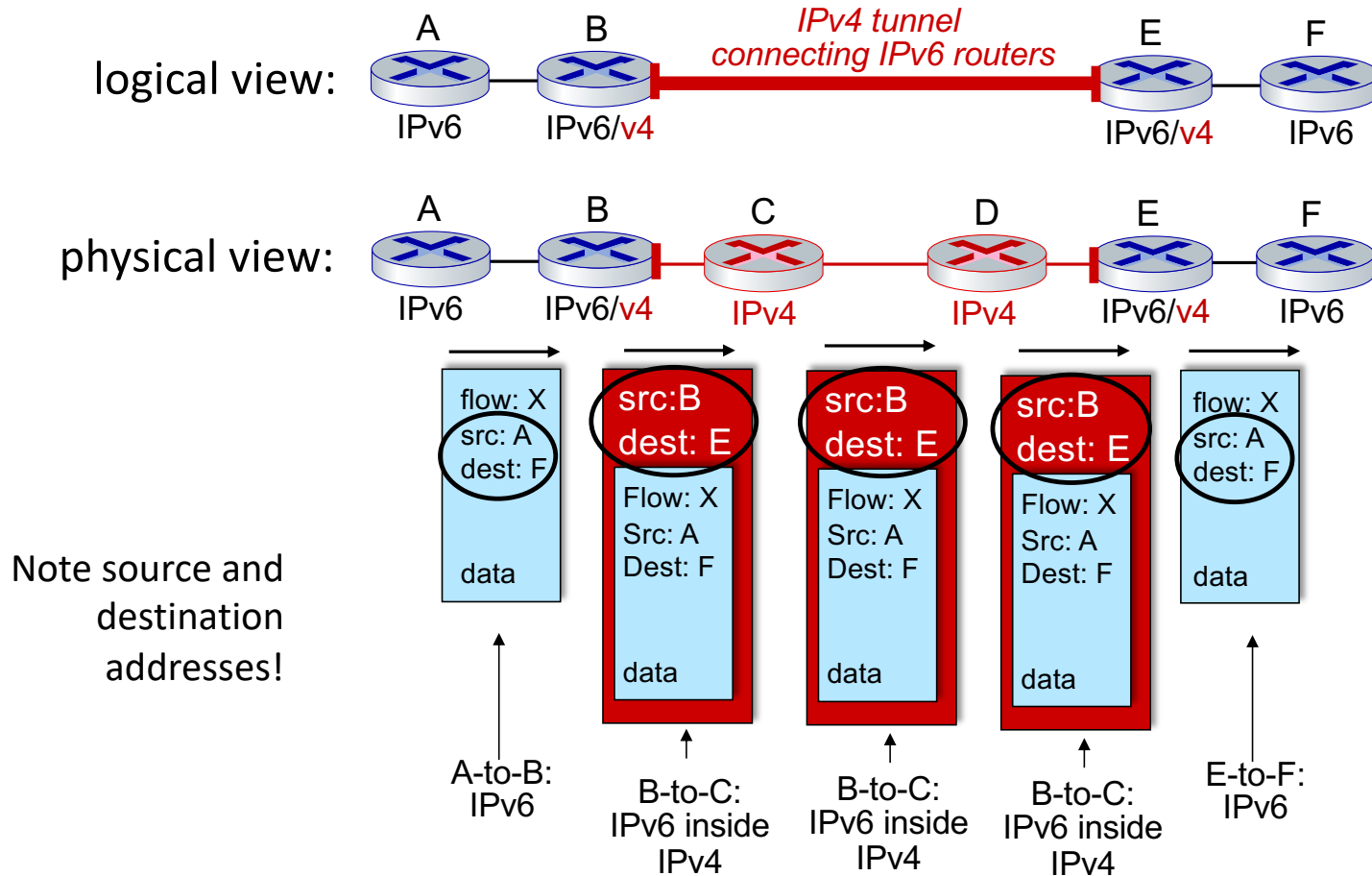
Ethernet connecting two IPv6 routers:



IPv4 tunnel connecting two IPv6 routers



Tunneling

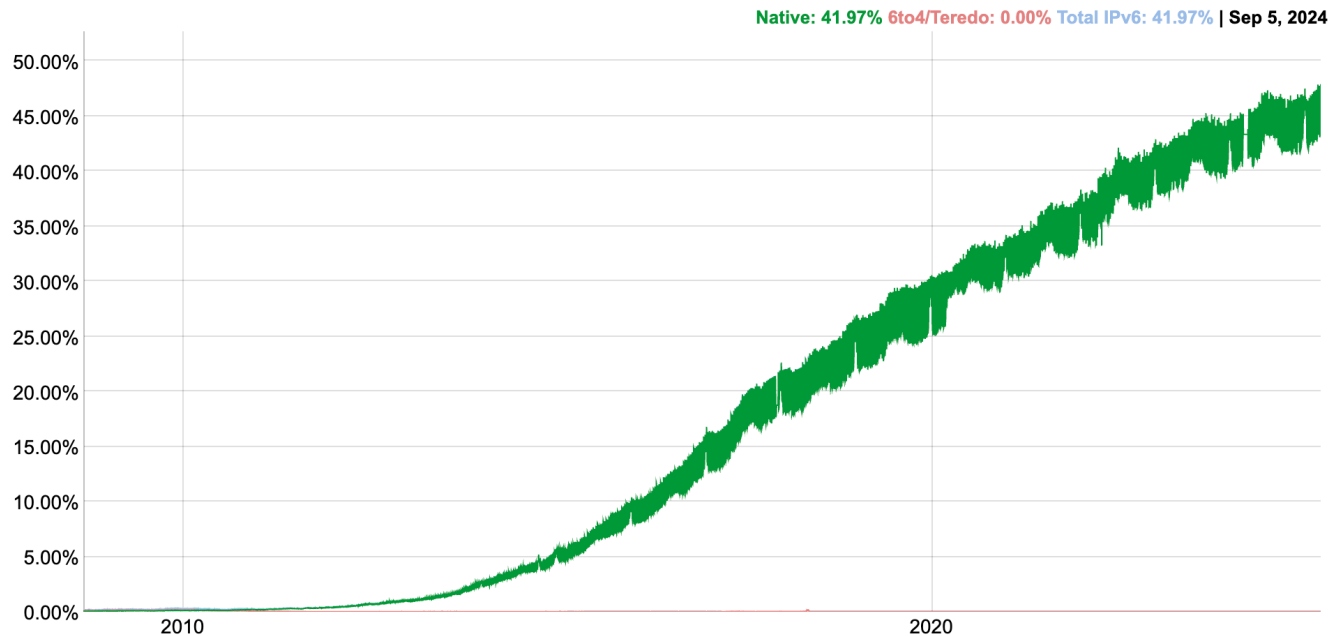


IPv6: adoption

- Google ¹ : 46~ 49% of clients access services via IPv6 (early 2026)
- NIST: 43% US government domains are IPv6 capable (early 2026)

IPv6 Adoption

We are continuously measuring the availability of IPv6 connectivity among Google users. The graph shows the percentage of users that access Google over IPv6.



¹ <https://www.google.com/intl/en/ipv6/statistics.html>

IPv6: adoption

- Google¹ : 46~ 49% of clients access services via IPv6 (early 2026)
- NIST: 43% US government domains are IPv6 capable (early 2026)
- Long (long!) time for deployment, use
 - 25 years and counting!
 - think of application-level changes in last 25 years: WWW, social media, streaming media, gaming, telepresence, ...
 - *Why?*

¹ <https://www.google.com/intl/en/ipv6/statistics.html>



CSEE4119: Computer Networks

Network Layer – Control Plane

Xia Zhou
Department of Computer Science
Columbia University

Slides credited to J.F Kurose and K.W. Ross and their textbook "Computer Networking: A Top-Down Approach"

Network layer: “control plane” roadmap

- introduction
- routing protocols
 - link state, distance vector
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol
- network management, configuration
 - SNMP, NETCONF/YANG

Network-layer functions

- **forwarding**: move packets from router's input to appropriate router output
- **routing**: determine route taken by packets from source to destination

data plane

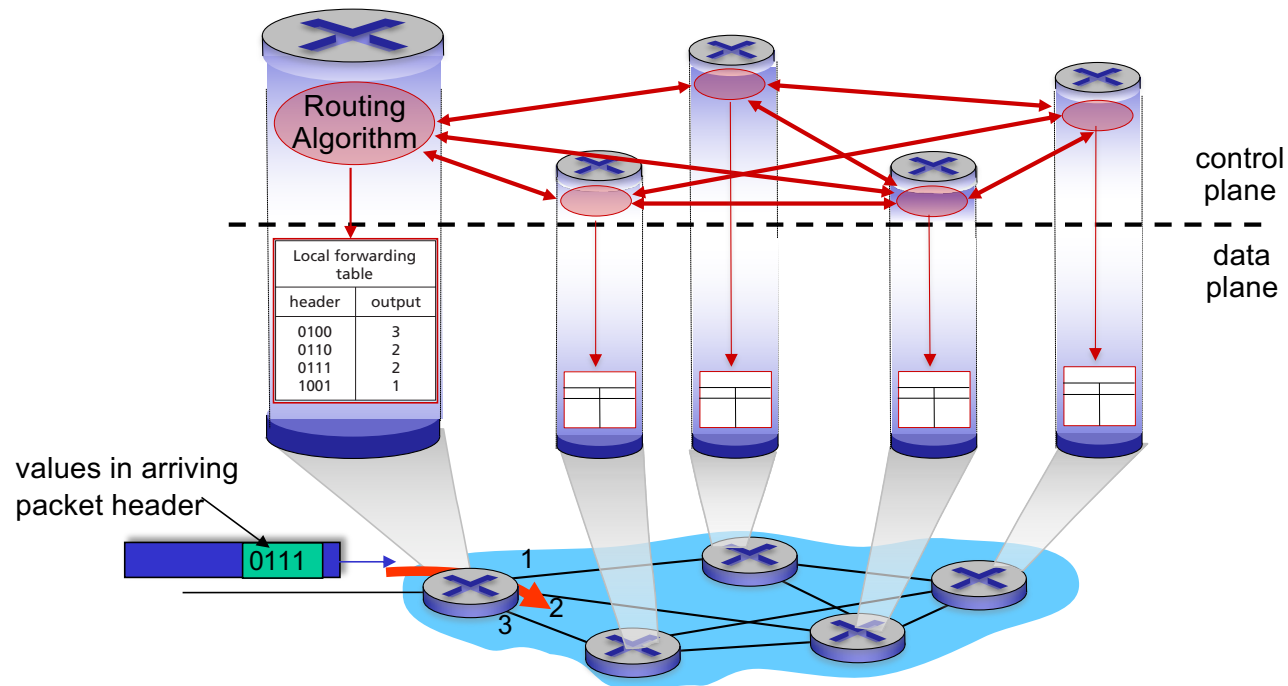
control plane

Two approaches to structuring network control plane:

- per-router control (traditional)
- logically centralized control (software defined networking)

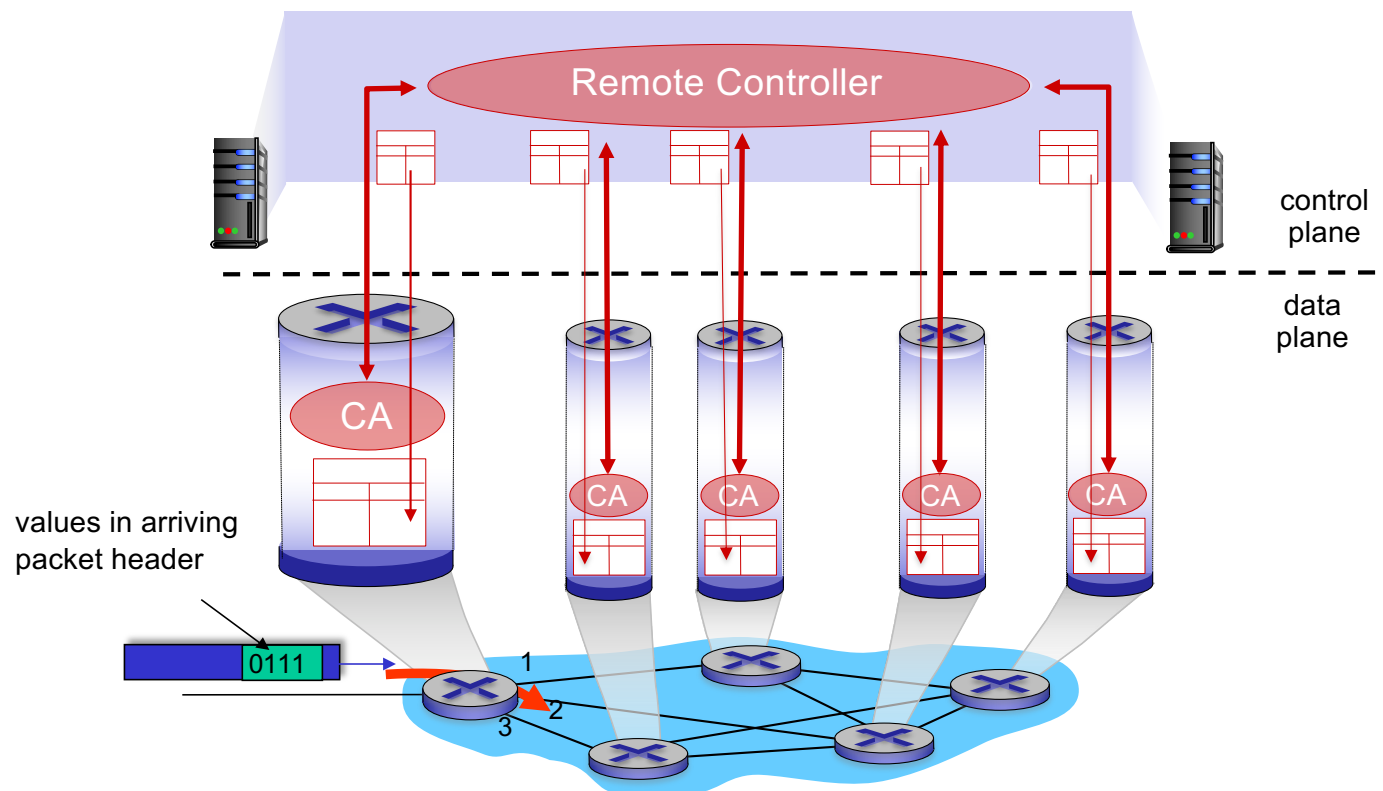
Per-router control plane

Individual routing algorithm components *in each and every router* interact in the control plane



Software-Defined Networking (SDN) control plane

Remote controller computes, installs forwarding tables in routers



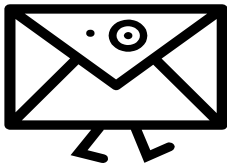
Network layer: “control plane” roadmap

- introduction
- routing protocols
 - link state, distance vector
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol
- network management, configuration
 - SNMP, NETCONF/YANG

What is routing?

- A quotation from RFC 791 says it the best

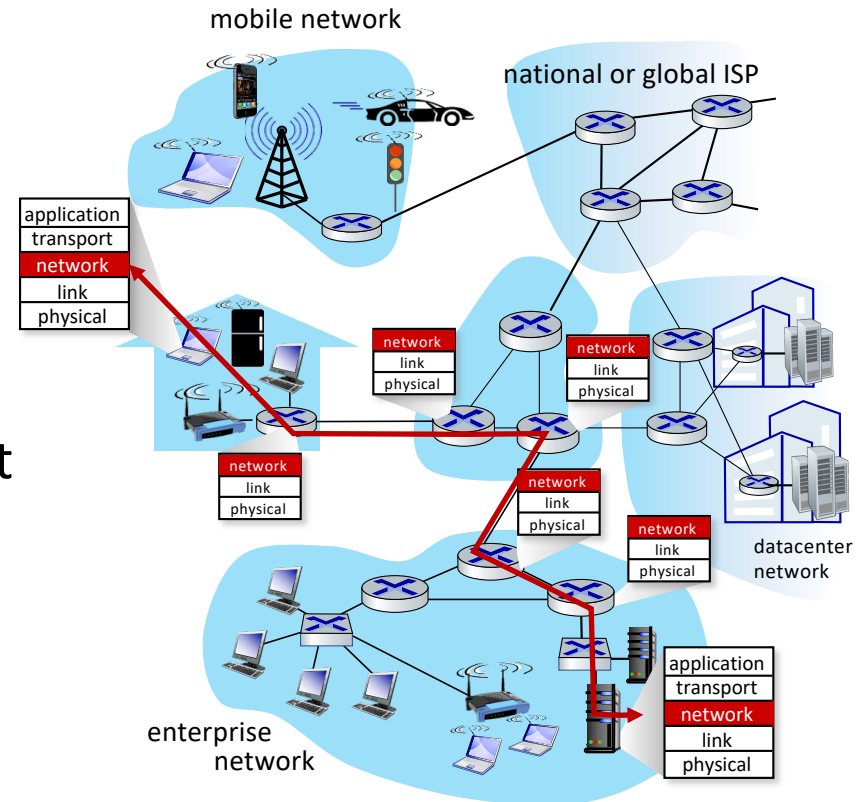
A name indicates what we seek.
An address indicates where it is.
A route indicates how we get there.
-- Jon Postel



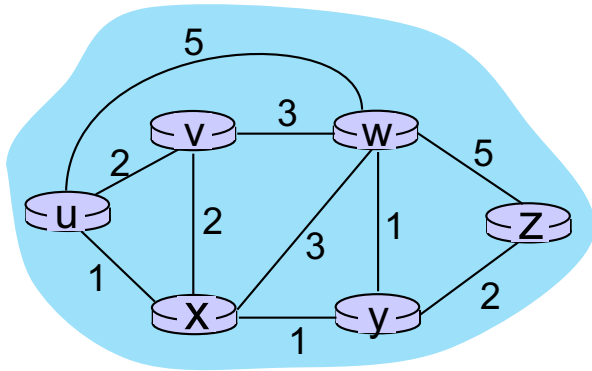
Routing protocols

Routing protocol goal: determine “good” paths (equivalently, routes), from sending hosts to receiving host, through network of routers

- **path:** sequence of routers packets traverse from given initial source host to destination host
- **“good”:** least “cost”, “fastest”, “least congested”
- routing: a “top-10” networking challenge!



Graph abstraction: link costs



$c_{a,b}$: cost of *direct* link connecting a and b
e.g., $c_{w,z} = 5$, $c_{u,z} = \infty$

cost defined by network operator:
depend on distance, bandwidth, cost,
congestion, and more

graph: $G = (N, E)$

N : set of routers = $\{ u, v, w, x, y, z \}$

E : set of links = $\{ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) \}$

Two Methodologies

■ Big mouth

Link State Routing

- Broadcast what you know to **all other nodes**
- Each node has a **global** view of the network
- Calculate paths based on the global network



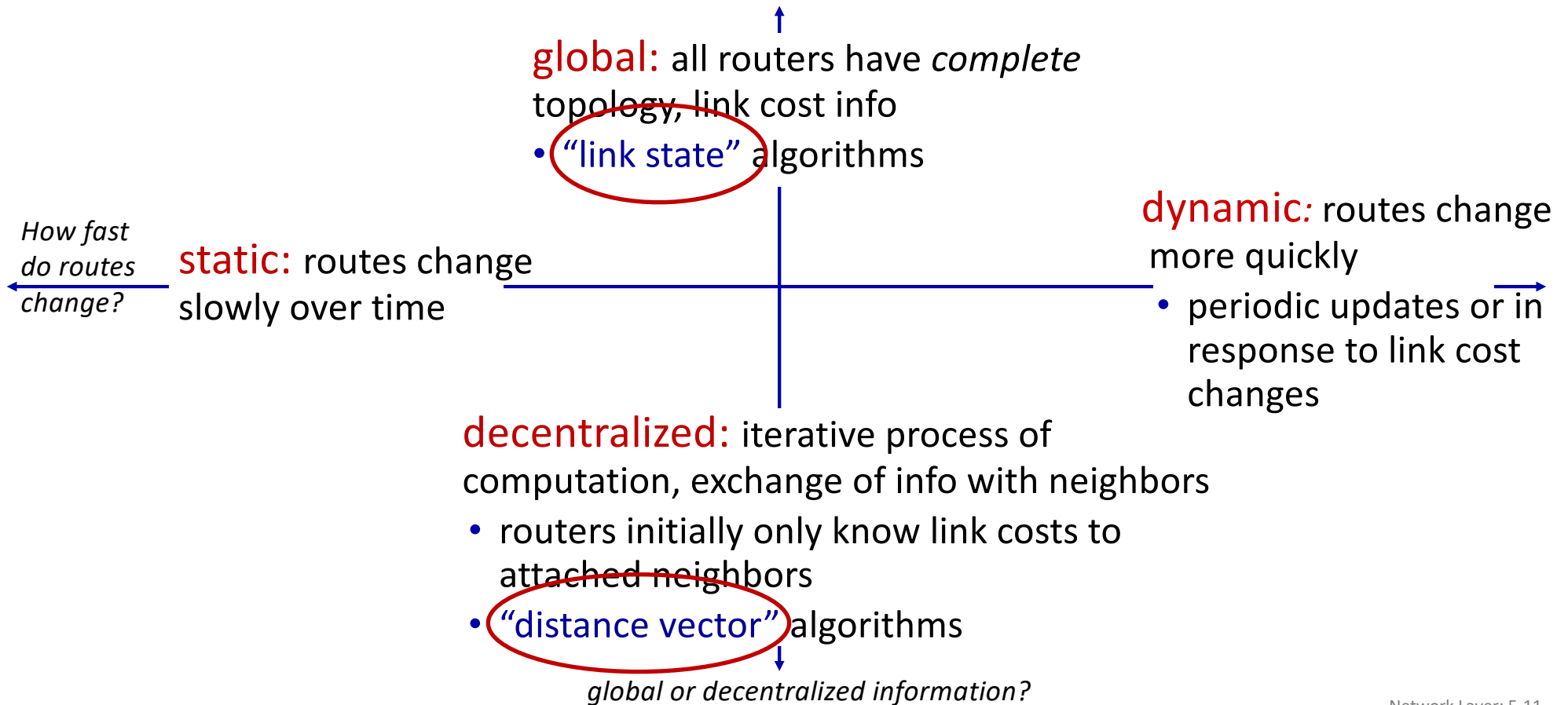
■ Whisper

Distance Vector Routing

- Only talk to **your neighbors** about what you know
- Each node has a **local** view of the network
- Update the path based on what you learnt from neighbors



Routing algorithm classification

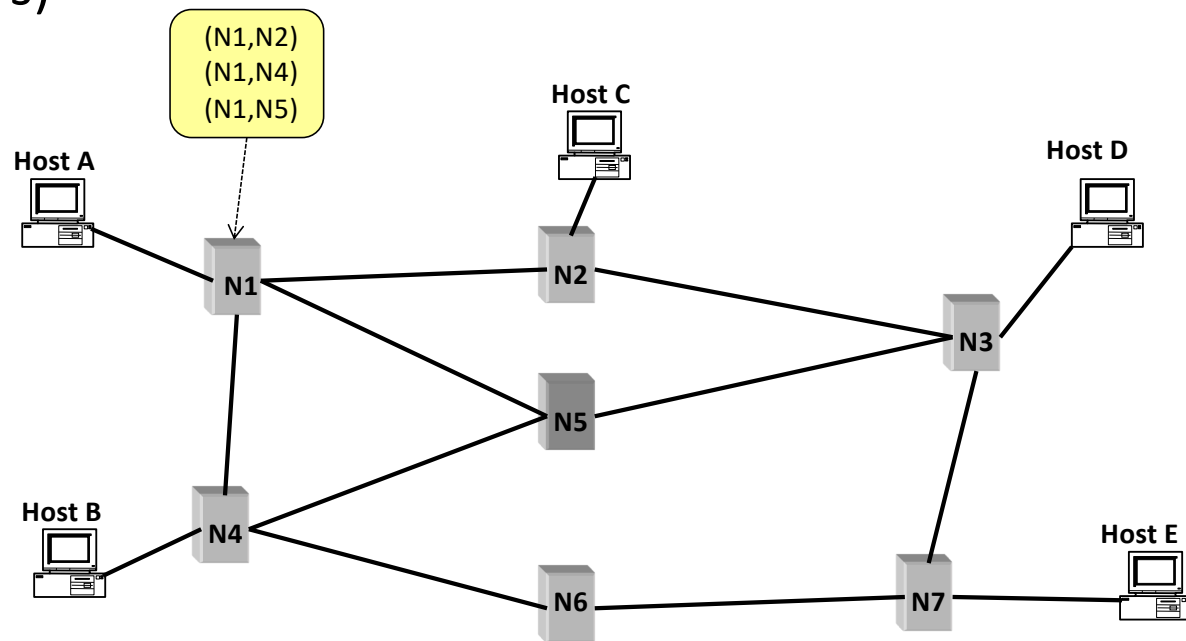


Network layer: “control plane” roadmap

- introduction
- routing protocols
 - link state
 - distance vector
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol
- network management, configuration
 - SNMP, NETCONF/YANG

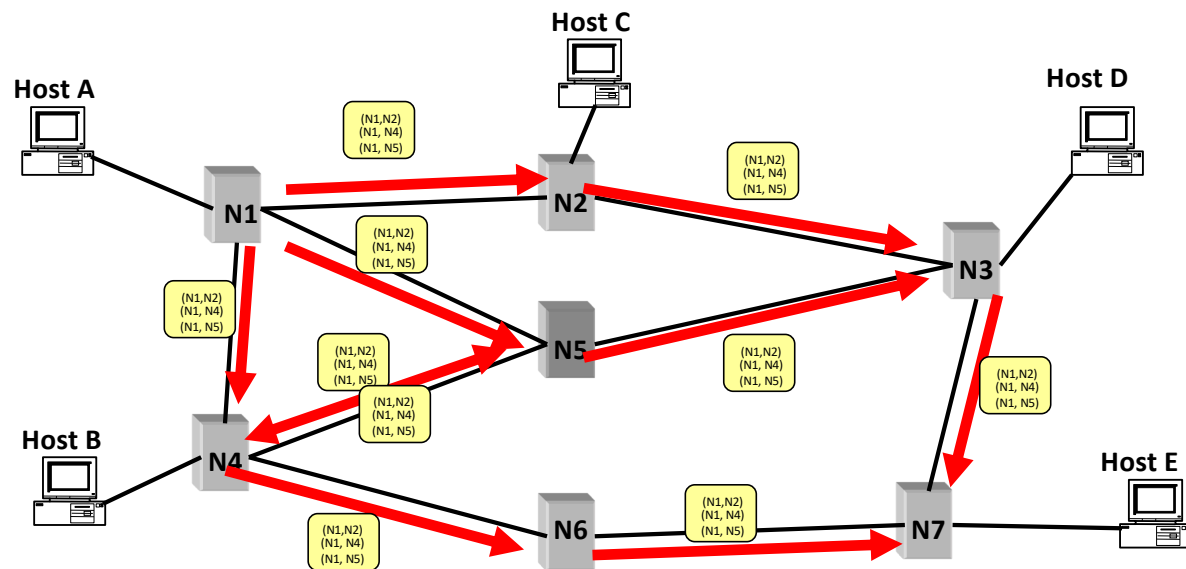
Link state routing

#1: Each node initially maintains a local link state (list of links to neighbors)



Link state routing

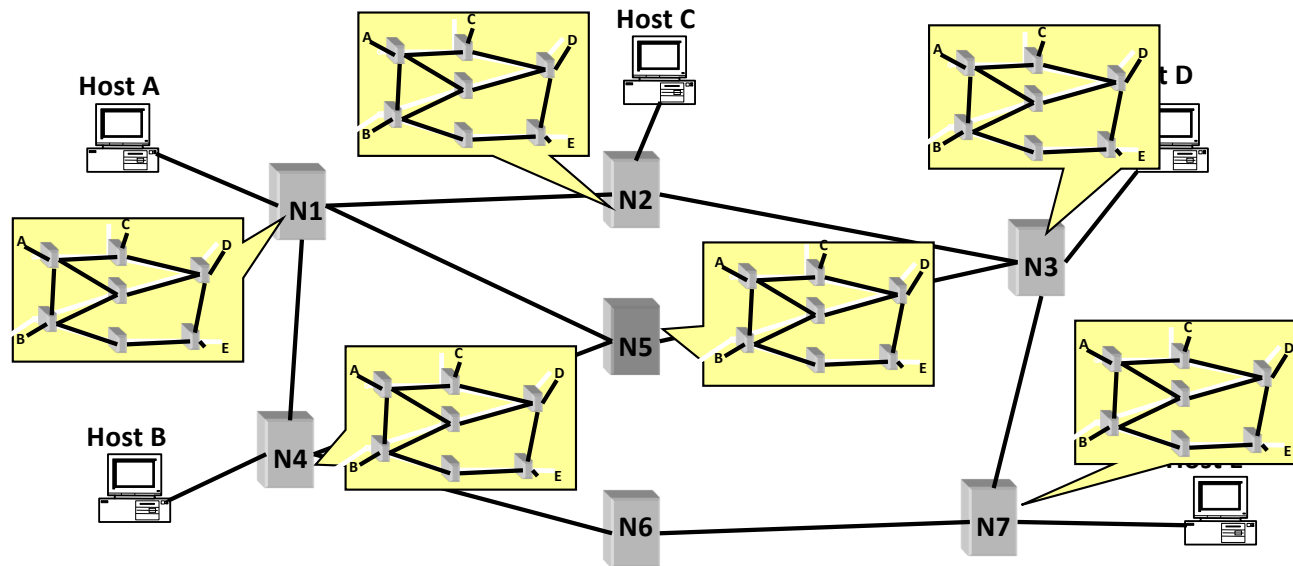
#2: Each node floods its local link state (other routers forward new info to all their neighbors)



Link state routing

#3: Each node learns the entire network topology

#4: Compute the shortest path using **Dijkstra's algorithm**



Dijkstra's link-state routing algorithm

- **centralized**: network topology, link costs known to *all* nodes
 - accomplished via “link state broadcast”
 - all nodes have same info
- computes least cost paths from one node (“source”) to all other nodes
 - gives *forwarding table* for that node
- **iterative**: after k iterations, know least cost path to k destinations

notation

- $c_{x,y}$: direct link cost from node x to y ; $= \infty$ if not direct neighbors
- $D(v)$: *current* estimate of cost of least-cost-path from source to destination v
- $p(v)$: predecessor node along path from source to v
- N' : set of nodes whose least-cost-path *definitively* known

Dijkstra's link-state routing algorithm

1 *Initialization:*

2 $N' = \{u\}$ /* compute least cost path from u to all other nodes */

3 for all nodes v

4 if v adjacent to u /* u initially knows direct-path-cost only to direct neighbors */

5 then $D(v) = c_{u,v}$ /* but may not be *minimum* cost! */

6 else $D(v) = \infty$

7

8 *Loop*

9 find w not in N' such that $D(w)$ is a minimum

10 add w to N'

11 update $D(v)$ for all v adjacent to w and not in N' :

12 **$D(v) = \min (D(v), D(w) + c_{w,v})$**

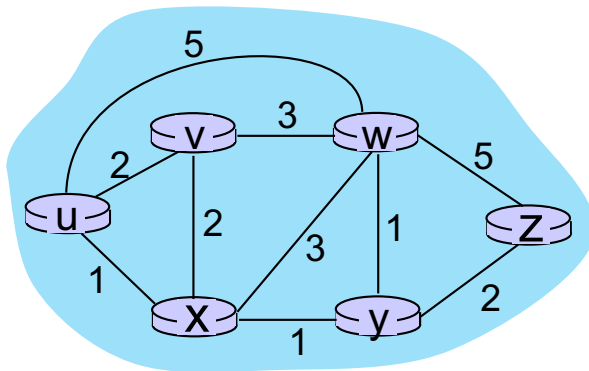
13 /* new least-path-cost to v is either old least-cost-path to v or known

14 least-cost-path to w plus direct-cost from w to v */

15 *until all nodes in N'*

Dijkstra's algorithm: an example

		v	w	x	y	z
Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1						
2						
3						
4						
5						

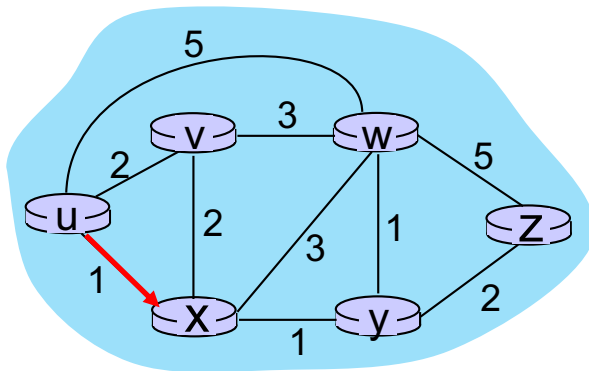


Initialization (step 0):

For all a : if a adjacent to u then $D(a) = c_{u,a}$

Dijkstra's algorithm: an example

Step	N'	^v D(v),p(v)	^w D(w),p(w)	^x D(x),p(x)	^y D(y),p(y)	^z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	u, x					
2						
3						
4						
5						

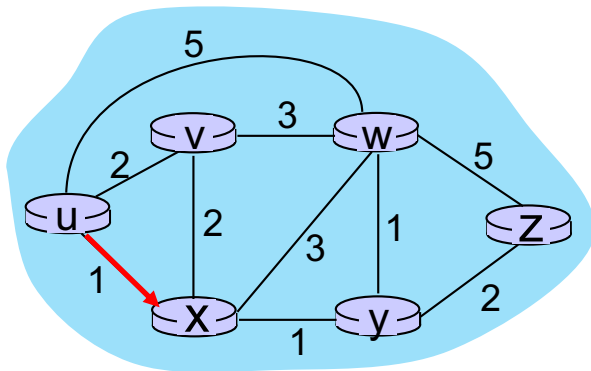


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: an example

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2						
3						
4						
5						



8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

$$D(v) = \min (D(v), D(x) + c_{x,v}) = \min(2, 1+2) = 2$$

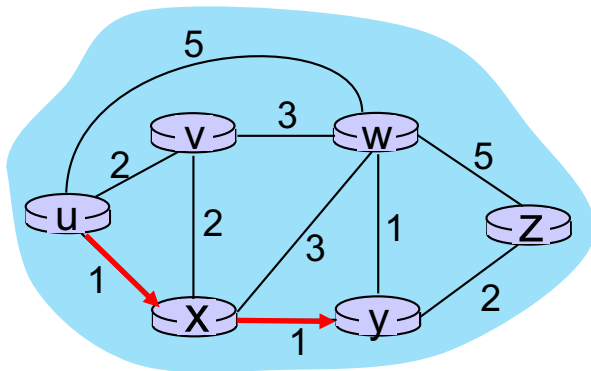
$$D(w) = \min (D(w), D(x) + c_{x,w}) = \min(5, 1+3) = 4$$

$$D(y) = \min (D(y), D(x) + c_{x,y}) = \min(\infty, 1+1) = 2$$



Dijkstra's algorithm: an example

Step	N'	^v D(v),p(v)	^w D(w),p(w)	^x D(x),p(x)	^y D(y),p(y)	^z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy					
3						
4						
5						

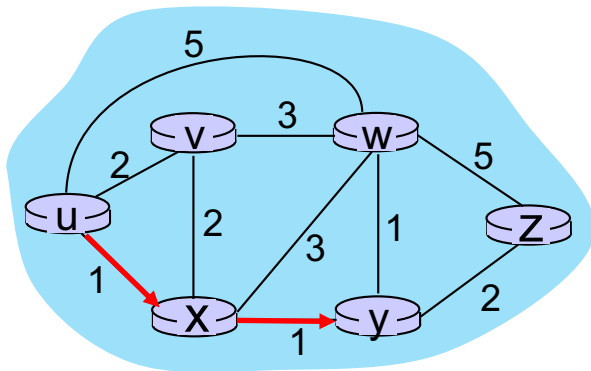


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: an example

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3						
4						
5						



8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

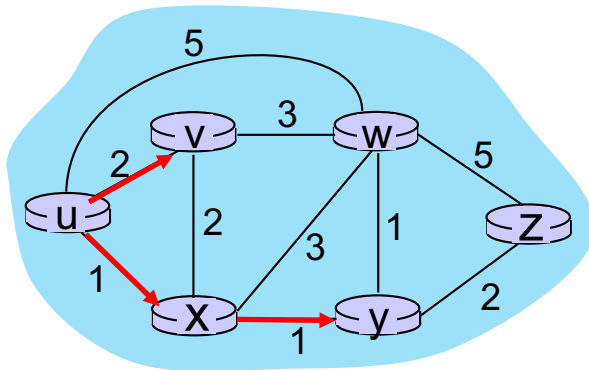
$$D(w) = \min (D(w), D(y) + c_{y,w}) = \min (4, 2+1) = 3$$

$$D(z) = \min (D(z), D(y) + c_{y,z}) = \min (\infty, 2+2) = 4$$



Dijkstra's algorithm: an example

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv					
4						
5						

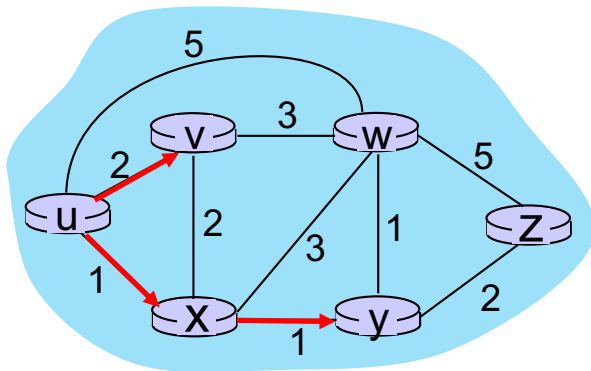


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: an example

Step	N'	^v D(v),p(v)	^w D(w),p(w)	^x D(x),p(x)	^y D(y),p(y)	^z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4						
5						



8 Loop

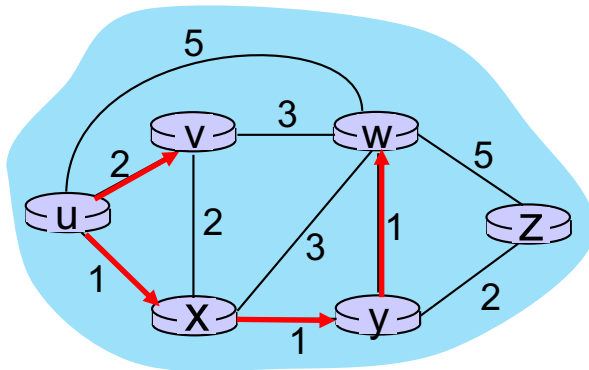
- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

$$D(w) = \min (D(w), D(v) + c_{v,w}) = \min (3, 2+3) = 3$$

Dijkstra's algorithm: an example

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					
5						

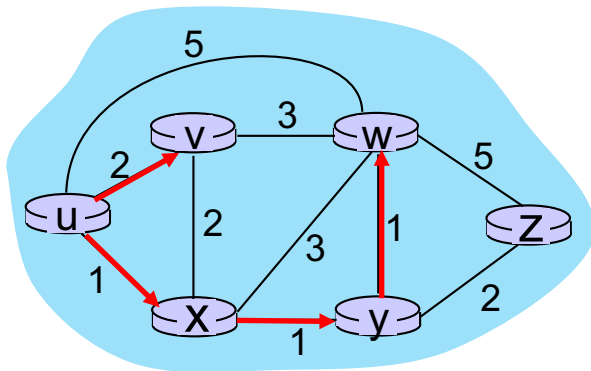


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: an example

Step	N'	v D(v),p(v)	w D(w),p(w)	x D(x),p(x)	y D(y),p(y)	z D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5						



8 Loop

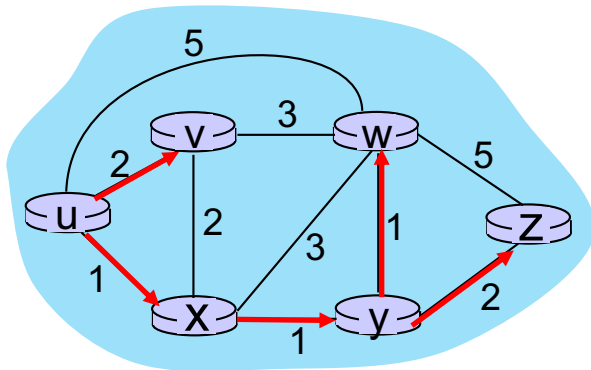
- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

$$D(z) = \min (D(z), D(w) + c_{w,z}) = \min (4, 3+5) = 4$$

Dijkstra's algorithm: an example

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					

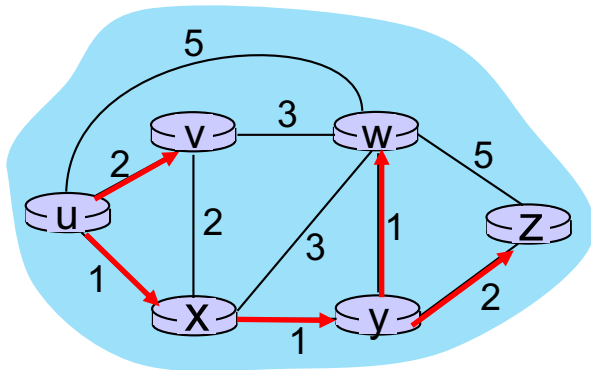


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: an example

		v	w	x	y	z
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					

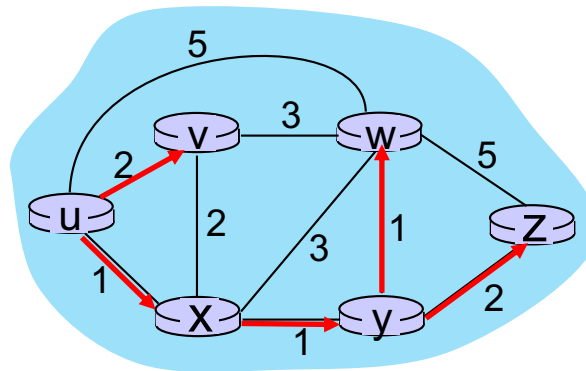


8 Loop

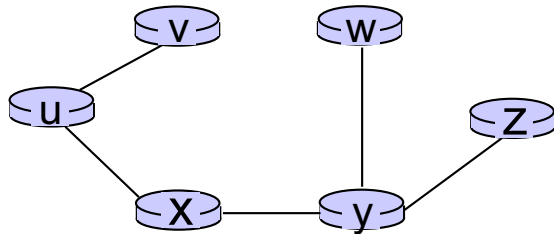
- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

Dijkstra's algorithm: an example



resulting least-cost-path tree from u:



resulting forwarding table in u:

destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
x	(u,x)

route from *u* to *v* directly

route from *u* to all other destinations via *x*

Dijkstra's algorithm: discussion

algorithm complexity: n nodes

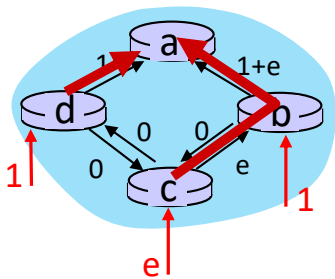
- each of n iteration: need to check all nodes, w , not in N
- $n(n+1)/2$ comparisons: $O(n^2)$ complexity
- more efficient implementations possible: $O(n \log n)$

message complexity:

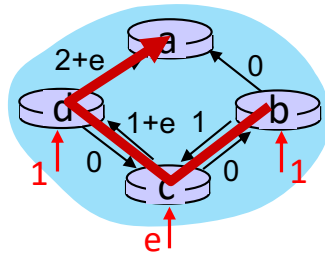
- each router must *broadcast* its link state information to other n routers
- efficient (and interesting!) broadcast algorithms: $O(n)$ link crossings to disseminate a broadcast message from one source
- each router's message crosses $O(n)$ links: overall message complexity: $O(n^2)$

Dijkstra's algorithm: oscillations possible

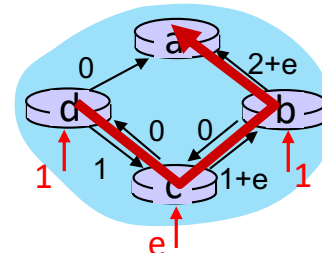
- when link costs depend on traffic volume, **route oscillations** possible
- sample scenario:
 - routing to destination a, traffic entering at d, c, b with rates 1, e (<1), 1
 - link costs are directional, and volume-dependent



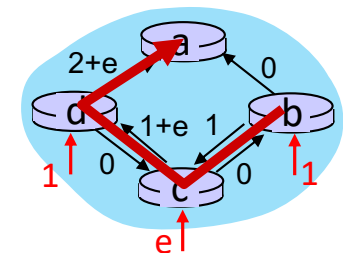
initially



given these costs,
find new routing....
resulting in new costs



given these costs,
find new routing....
resulting in new costs



given these costs,
find new routing....
resulting in new costs

Two Methodologies

■ Big mouth

Link State Routing

- Broadcast what you know to **all other nodes**
- Each node has a **global** view of the network
- Calculate paths based on the global network



■ Whisper

Distance Vector Routing

- Only talk to **your neighbors** about what you know
- Each node has a **local** view of the network
- Update the path based on what you learnt from neighbors



Time for a game!



- The question: Who carries the most cash today?
- Rule
 - ONLY talk to your **neighbors** sitting left or right to you
 - No talking to others
 - No shouting
 - No gesture or other innovative ways of notifying non-neighbors
- After 3mins, I will randomly pick one of you to answer the question
- Ready? Go!

Distance vector algorithm

Based on *Bellman-Ford* (BF) equation (dynamic programming):

Bellman-Ford equation

Let $D_x(y)$: cost of least-cost path from x to y .

Then:

$$D_x(y) = \min_v \{ c_{x,v} + D_v(y) \}$$

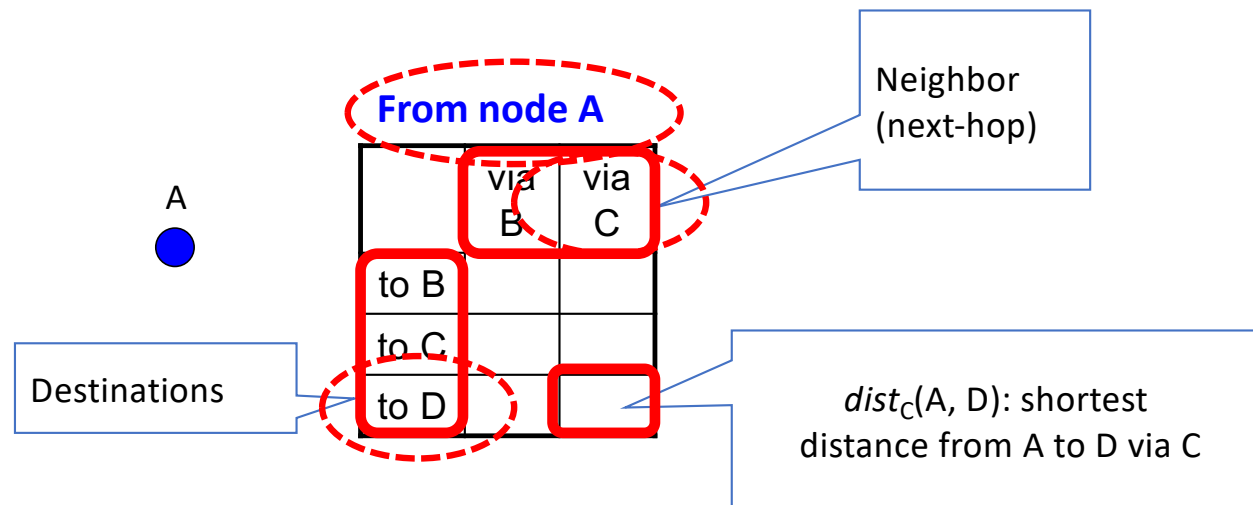
$\min$ taken over all neighbors v of x

direct cost of link from x to v

v 's estimated least-cost-path cost to y

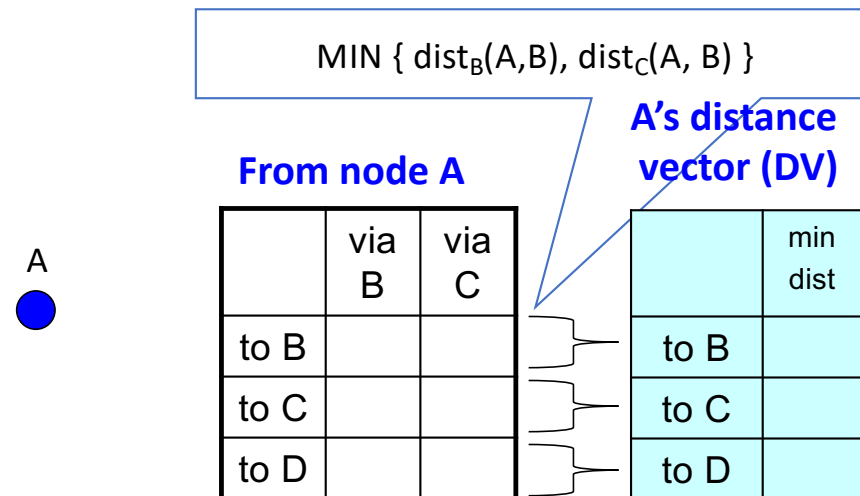
Distance vector table

- Each router maintains its shortest distance to each other node by going through each neighbor



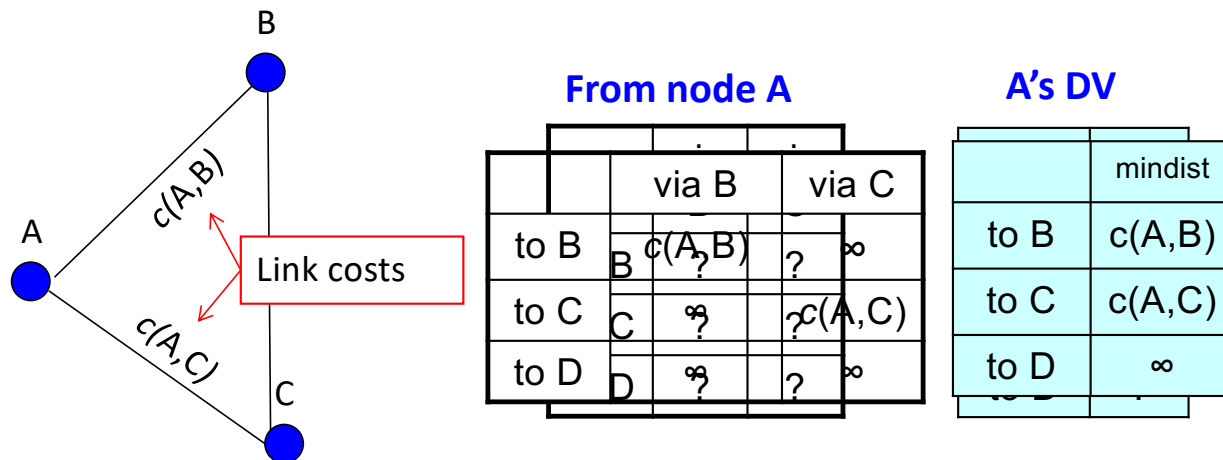
Distance vector

- Each router computes the shortest distance to each other node



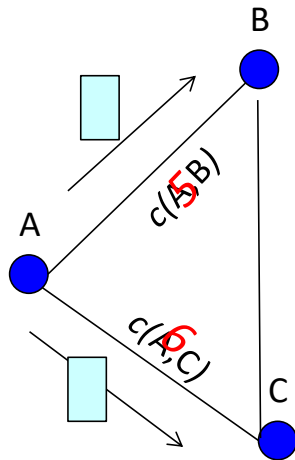
Distance vector

- Initialize the distance table and DV based on its neighbors' link costs



Distance vector

- Send its DV to neighbors



From node A

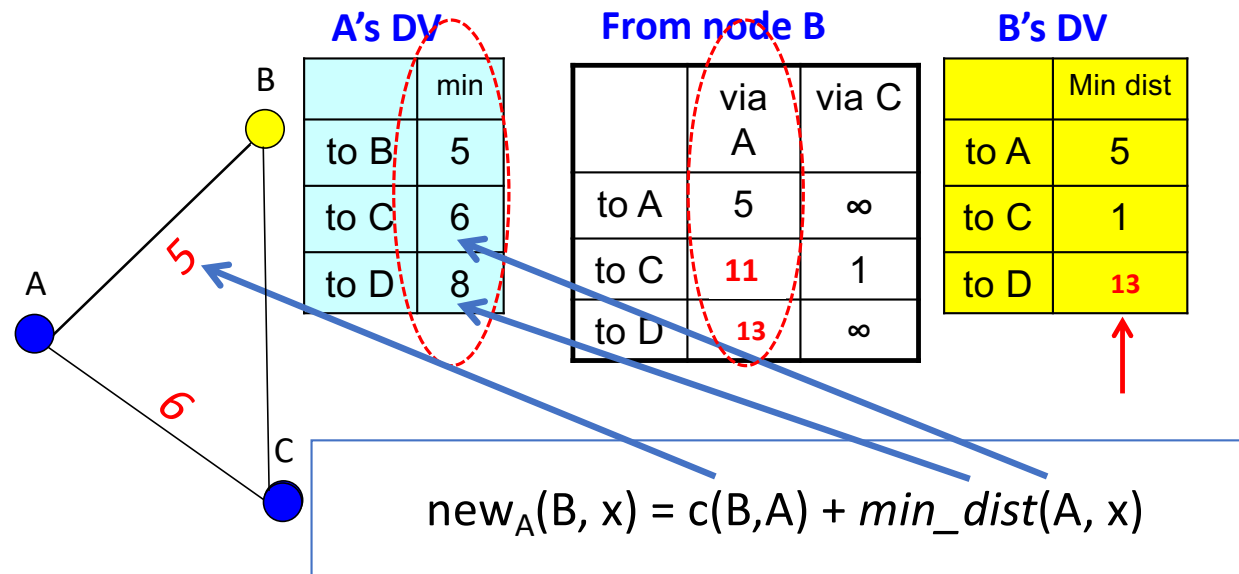
	via B	via C
to B	$c(A,B)$	∞
to C	∞	$c(A,C)$
to D	∞	8

A's DV

	Min dist
to B	5
to C	6
to D	8

Distance vector

- Neighbor router processes received DVs



Distance vector algorithm

key idea:

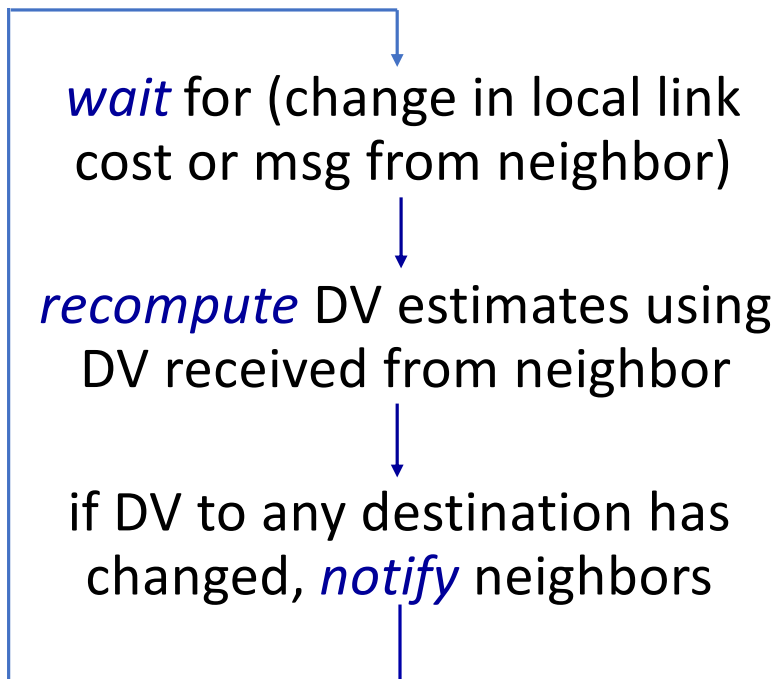
- from time-to-time, each node sends its own distance vector estimate to neighbors
- when x receives new DV estimate from any neighbor, it updates its own DV using B-F equation:

$$D_x(y) \leftarrow \min_v \{c_{x,v} + D_v(y)\} \text{ for each node } y \in N$$

- under minor, natural conditions, the estimate $D_x(y)$ converge to the actual least cost $d_x(y)$

Distance vector algorithm:

each node:



iterative, asynchronous: each local iteration caused by:

- local link cost change
- DV update message from neighbor

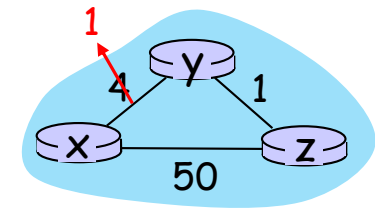
distributed, self-stopping: each node notifies neighbors *only* when its DV changes

- neighbors then notify their neighbors – *only if necessary*
- no notification received, no actions taken!

Distance vector: link cost changes

link cost changes:

- node detects local link cost change
- updates routing info, recalculates local DV
- if DV changes, notify neighbors



Y via			Y via			Y via			Y via		
	X	Z		X	Z		X	Z		X	Z
X	4	6	X	1	6	X	1	6	X	1	3

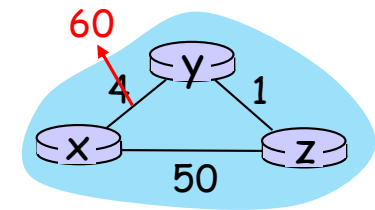
Z via			Z via			Z via			Z via		
	X	Y		X	Y		X	Y		X	Y
X	50	5	X	50	5	X	50	2	X	50	2

Good news goes fast!

Distance vector: link cost changes

link cost changes:

- node detects local link cost change
- updates routing info, recalculates local DV
- if DV changes, notify neighbors



Y via			Y via			Y via			Y via		
	X	Z		X	Z		X	Z		X	Z
X	4	6	X	60	6	X	60	6	X	60	8

Z via			Z via			Z via			Z via		
	X	Y		X	Y		X	Y		X	Y
X	50	5	X	50	5	X	50	7	X	50	7

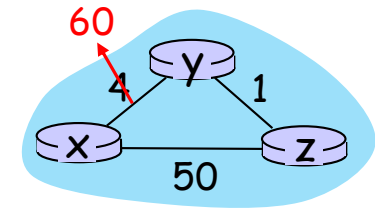
Count to infinity problem

The process goes on....

Bad news goes slow...

Distance vector: poison reverse

- If Z routes through Y to get to X :
 - Z tells Y its (Z's) distance to X is **infinite** (so Y won't route to X via Z)



Y via			Y via			Y via			Y via			Y via		
	X	Z		X	Z		X	Z		X	Z		X	Z
X	4	∞	X	60	∞	X	60	∞	X	60	51	X	60	51

Z via			Z via			Z via			Z via			Z via		
	X	Y		X	Y		X	Y		X	Y		X	Y
X	50	5	X	50	5	X	50	61	X	50	61	X	50	∞

Comparison of LS and DV algorithms

message complexity

LS: n routers, $O(n^2)$ messages sent

DV: exchange between neighbors;
convergence time varies

speed of convergence

LS: $O(n^2)$ algorithm, $O(n^2)$ messages

- may have oscillations

DV: convergence time varies

- may have routing loops
- count-to-infinity problem

robustness: what happens if router malfunctions, or is compromised?

LS:

- router can advertise incorrect *link* cost
- each router computes only its *own* table

DV:

- DV router can advertise incorrect *path* cost (“I have a *really* low-cost path to everywhere”): *black-holing*
- each router’s DV is used by others: error propagate thru network